

Analisis Perbandingan Strategi BFS, A*, dan Beam Search pada Navigasi Agen dalam Lingkungan Wumpus World

Haikal Imam Ridha - 24060124130097
Departemen Informatika
Universitas Diponegoro
Semarang, Indonesia
haikalimamridha@students.undip.ac.id

Abstract— Pencarian jalur yang aman dan efisien merupakan tantangan utama dalam navigasi agen cerdas pada lingkungan Wumpus World. Penelitian ini bertujuan untuk melakukan analisis komparatif terhadap performa tujuh algoritma pencarian, yaitu Greedy Search, Depth-First Search (DFS), Breadth-First Search (BFS), Uniform-Cost Search (UCS), Greedy Best-First Search (GBFS), A*, dan Beam Search ($\beta=3$). Pengujian dilakukan pada lingkungan Wumpus World termodifikasi berukuran 8×8 tanpa mekanis persepsi breeze dan stench, di mana algoritma berbasis informatif menggunakan heuristik jarak Manhattan. Parameter evaluasi yang digunakan meliputi panjang jalur, jumlah node yang dieksplorasi, dan waktu eksekusi. Hasil pengujian menunjukkan bahwa BFS, UCS, A*, dan Beam Search berhasil menemukan jalur optimal terpendek (14 langkah). GBFS menjadi algoritma tercepat (0,0928 ms) dan paling efisien dalam eksplorasi node (17 node), namun menghasilkan jalur sub-optimal (16 langkah). Sebaliknya, DFS menghasilkan performa terburuk dengan jalur sepanjang 34 langkah. Penelitian ini menyimpulkan bahwa A* dan Beam Search memberikan kompromi terbaik antara optimalitas solusi dan efisiensi komputasi untuk navigasi agen.

Keywords— *Wumpus World, Algoritma Pencarian, Analisis Komparatif, Jarak Manhattan, Efisiensi Navigasi.*

I. PENDAHULUAN

Salah satu permasalahan utama dalam AI adalah pencarian(search), yaitu proses untuk menemukan urutan yang tepat untuk mencapai tujuan tertentu dari kondisi awal. Algoritma pencarian banyak digunakan pada berbagai bidang, seperti robotika, sistem navigasi, manajemen data, dan lainnya [1].

Salah satu lingkungan yang sering digunakan untuk mempelajari perilaku agen cerdas adalah wumpus. Wumpus menerapkan permasalahan pencarian(search) dengan tujuan agen dapat menemukan sebuah emas dengan menghindari rintangan bahaya berupa lubang(pit) dan monster wumpus. Agen harus dapat menentukan jalur yang aman dan efisien berdasarkan informasi yang diperoleh [1].

Berbagai macam strategi pencarian dapat diterapkan dalam navigasi agen pada wumpus world. Dalam makalah ini akan diterapkan beberapa strategi pencarian, diantaranya yaitu Greedy, Depth-First Search(DFS), Breadth-First Search(BFS), Uniform-Cost Search(UCS), Greedy Best First Search(Greedy BFS), A*, dan Beam Search.

Masing masing algoritma memiliki kelebihan dan kekurangan dalam waktu pencarian, penggunaan memori, serta kualitas solusi yang dihasilkan. Oleh karena itu diperlukan analisis komparatif untuk membandingkan performa dari beberapa algoritma tersebut ketika digunakan dalam navigasi lingkungan di lingkungan wumpus world. Hasil analisis diharapkan dapat memberikan gambaran

mengenai perbandingan beberapa algoritma ketika digunakan dalam navigasi agen.

II. BATASAN MASALAH

Penelitian ini memiliki beberapa batasan masalah, yaitu:

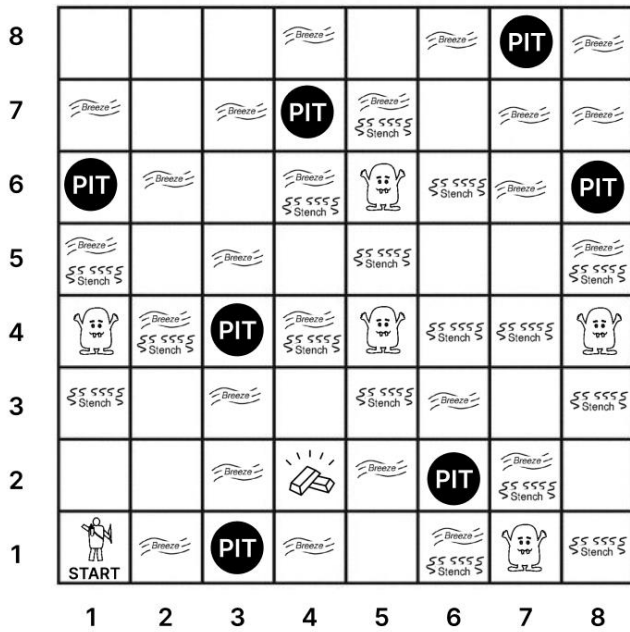
1. Lingkungan yang digunakan adalah wumpus world 8×8
2. Algoritma yang dibandingkan yaitu Greedy, DFS, BFS, UCS, Greedy BFS, A*, dan Beam Search
3. Posisi agen, tujuan, lubang(pit), dan wumpus telah diketahui oleh sistem sehingga penelitian berfokus pada perbandingan algoritma pencarian jalur, bukan pada proses inferensi pengetahuan.
4. Wumpus world dibuat tidak menggunakan breeze dan stench
5. Agen hanya dapat bergerak ke atas, bawah, kanan, dan kiri
6. Algoritma A*, Greedy BFS, dan Beam Search menggunakan nilai heuristik jarak manhattan
7. Nilai beam width pada Beam Search ditentukan sebelum proses pengujian dan tidak berubah selama pencarian.
8. Penelitian tidak membahas penerapan machine learning, reinforcement learning, maupun inferensi logika Wumpus World.
9. Evaluasi kinerja dilakukan berdasarkan waktu eksekusi, jumlah node yang dieksplorasi, dan tingkat keberhasilan menemukan tujuan

III. DASAR TEORI

A. Wumpus World

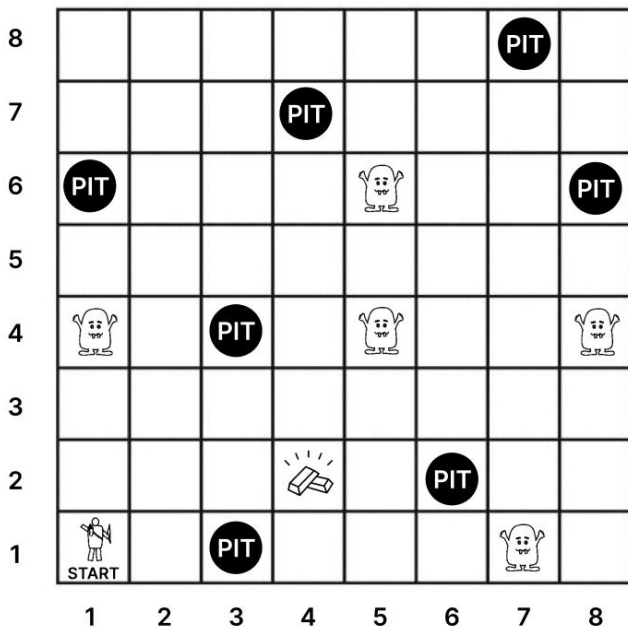
Lingkungan wumpus world adalah grid persegi yang dikelilingi dinding, di mana setiap persegi dapat berisi agen dan objek. Ketika agen berada di kotak yang berdekatan langsung dengan wumpus (bukan diagonal), agen akan merasakan stench. Dan ketika agen berada di kotak yang berdekatan langsung dengan pit, agen akan merasakan breeze [1].

Agen selalu memulai di sudut kiri bawah, sebuah persegi yang akan kita beri label [1,1]. Tugas agen adalah menemukan emas, kembali ke [1,1] dan memanjat keluar dari gua [1]. Contoh wumpus world ditunjukkan pada Gambar 3.1.



Gambar 3.1.

Namun di dalam makalah ini, stench dan breeze akan dihilangkan karena sistem sudah mengetahui semua letak pit dan wumpus sehingga keberadaan breeze dan stench tidak berpengaruh terhadap algoritma. Ilustrasi wumpus world pada Gambar 3.1. yang sudah dimodifikasi dapat dilihat pada Gambar 3.2.



Gambar 3.2.

B. Greedy

Algoritma greedy adalah sebuah paradigma desain algoritma yang menyelesaikan masalah optimasi dengan membuat pilihan yang tampaknya terbaik pada setiap langkah saat itu juga [2].

Strategi ini mengambil pilihan terbaik lokal (locally optimal choice) dengan harapan bahwa pilihan tersebut akan

mengarah pada solusi terbaik global (globally optimal solution).

C. Depth-First Search

Depth-First Search (DFS) menjelajahi sisi sisi dari simpul v yang baru ditemukan dan masih memiliki sisi sisi yang belum dijelajahi. Setelah semua sisi dijelajahi, akan dilakukan backtrack untuk menjelajahi sisi sisi yang keluar dari simpul tempat v ditemukan. Proses ini berlanjut sampai simpul yang dicari ditemukan [2].

D. Breadth-First Search

Breadth-First Search (BFS) merupakan algoritma pencarian graf yang menjelajahi simpul berdasarkan tingkat kedalaman dari simpul sumber. BFS menemukan semua simpul yang dapat dijangkau dengan menelusuri tetangga terdekat terlebih dahulu sebelum melanjutkan ke tingkat berikutnya. Algoritma ini menggunakan struktur data *queue* (FIFO) untuk mengelola simpul yang akan dieksplorasi dan membangun pohon pencarian yang memuat seluruh simpul yang dapat dicapai. Pada graf tanpa bobot atau dengan biaya seragam, BFS mampu menemukan jalur terpendek dari simpul sumber ke simpul tujuan berdasarkan jumlah sisi yang dilalui [2].

E. Uniform Cost Search

Uniform Cost Search (UCS) merupakan algoritma pencarian yang mengeksplorasi simpul berdasarkan biaya lintasan kumulatif terkecil dari simpul awal. Pada setiap langkah, algoritma memilih simpul dengan biaya terendah yang belum dieksplorasi menggunakan struktur data priority queue. Proses ini berlanjut hingga simpul tujuan ditemukan atau seluruh simpul yang dapat dijangkau telah dieksplorasi. Dengan asumsi seluruh biaya bernilai non-negatif, UCS mampu menemukan solusi optimal karena selalu mengembangkan lintasan dengan biaya terendah terlebih dahulu. Konsep UCS pada penelitian ini mengacu pada prinsip pencarian biaya minimum yang setara dengan mekanisme pemilihan simpul pada algoritma Dijkstra [2].

F. Greedy Best-First Search

Greedy Best-First Search (GBFS) merupakan algoritma pencarian berinformasi yang memilih simpul untuk dieksplorasi berdasarkan nilai heuristik yang memperkirakan kedekatan simpul terhadap tujuan. Algoritma ini menggunakan fungsi evaluasi

$$f(n) = h(n)$$

di mana $h(n)$ menyatakan estimasi biaya dari simpul saat ini menuju tujuan. Pada setiap langkah, simpul dengan nilai heuristik terkecil akan dipilih untuk diekspansi terlebih dahulu. Pendekatan ini memungkinkan pencarian mengarah langsung ke tujuan sehingga sering kali lebih cepat dibandingkan algoritma pencarian tidak berinformasi, namun tidak menjamin diperolehnya jalur optimal karena tidak mempertimbangkan biaya lintasan yang telah ditempuh [1].

G. A*

A* merupakan algoritma pencarian berinformasi yang menggabungkan biaya lintasan yang telah ditempuh dengan

estimasi biaya menuju tujuan untuk menentukan simpul yang akan dieksplorasi. Algoritma ini menggunakan fungsi evaluasi

$$f(n) = g(n) + h(n)$$

$g(n)$: biaya aktual dari titik awal menuju node n .

$h(n)$: estimasi biaya dari node n menuju tujuan.

$f(n)$: total estimasi biaya solusi melalui node n .

Pada setiap langkah, A* memilih simpul dengan nilai $f(n)$ terkecil untuk diekspansi. Dengan menggunakan heuristik yang admissible dan konsisten, A* mampu menemukan solusi optimal sekaligus mengurangi jumlah simpul yang perlu dieksplorasi dibandingkan metode pencarian tanpa informasi [1].

H. Beam Search

Beam Search merupakan algoritma pencarian heuristik yang mempertahankan sejumlah terbatas simpul terbaik pada setiap tahap pencarian. Algoritma ini bekerja dengan menyimpan k simpul yang dianggap paling menjanjikan berdasarkan nilai evaluasi atau heuristik yang digunakan. Pada setiap iterasi, seluruh simpul penerus dari k simpul tersebut dihasilkan dan dievaluasi, kemudian hanya k simpul terbaik yang dipilih untuk dilanjutkan pada proses pencarian berikutnya. Dengan membatasi jumlah simpul yang disimpan, Beam Search dapat mengurangi kebutuhan memori dan mempercepat proses pencarian. Namun, karena beberapa kandidat dapat dieliminasi lebih awal, algoritma ini tidak menjamin ditemukannya solusi optimal maupun solusi jika jalur yang benar telah terbuang pada tahap sebelumnya [1].

III. METODOLOGI

Penelitian ini dilakukan melalui beberapa tahapan, yaitu perancangan lingkungan Wumpus World, implementasi algoritma pencarian, pelaksanaan pengujian, pengumpulan data hasil pengujian, serta analisis dan perbandingan kinerja algoritma.

Lingkungan Wumpus World dimodelkan dalam bentuk grid berukuran 8×8 . Setiap sel pada grid dapat berisi ruang kosong, pit, Wumpus, posisi awal agen (start), atau gold sebagai tujuan (goal). Dalam penelitian ini, pit dan Wumpus diperlakukan sebagai obstacle yang tidak dapat dilalui oleh agen. Agen hanya dapat bergerak ke empat arah, yaitu atas, bawah, kiri, dan kanan. Mekanisme persepsi berupa breeze dan stench tidak digunakan karena penelitian difokuskan pada analisis performa algoritma pencarian jalur.

Penelitian ini mengimplementasikan tujuh algoritma pencarian, yaitu Depth-First Search (DFS), Breadth-First Search (BFS), Uniform Cost Search (UCS), Greedy Search, Greedy Best-First Search (GBFS), A*, dan Beam Search. Seluruh algoritma menerima masukan berupa lingkungan Wumpus World yang sama sehingga hasil pengujian dapat dibandingkan secara adil. Algoritma Greedy Search, GBFS, A*, dan Beam Search menggunakan heuristik Manhattan Distance untuk memperkirakan jarak dari posisi agen menuju tujuan. Rumus heuristiknya yaitu

$$h(n) = |x_1 - x_2| + |y_1 - y_2|$$

Kinerja algoritma dievaluasi menggunakan beberapa parameter sebagai berikut:

1. Panjang Jalur

Jumlah langkah yang diperlukan untuk mencapai tujuan.

2. Jumlah Node dieksplorasi

Banyaknya node yang dikunjungi selama proses pencarian.

3. Waktu eksekusi

Waktu yang dibutuhkan algoritma untuk menemukan solusi.

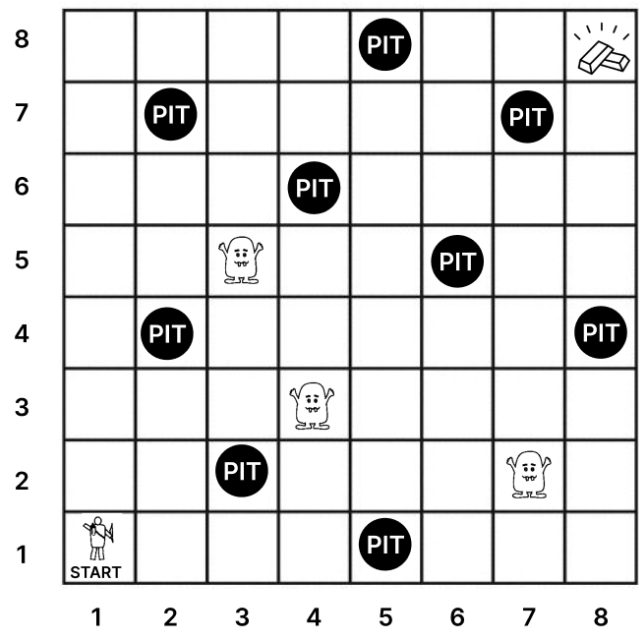
4. Tingkat keberhasilan

Kemampuan algoritma menemukan jalur menuju tujuan.

Data hasil pengujian dianalisis dengan membandingkan nilai panjang jalur, jumlah node yang dieksplorasi, waktu eksekusi, dan tingkat keberhasilan dari setiap algoritma. Hasil analisis digunakan untuk mengidentifikasi kelebihan dan kekurangan masing-masing algoritma dalam menyelesaikan permasalahan navigasi pada lingkungan Wumpus World.

IV. HASIL DAN PEMBAHASAN

Pengujian dilakukan pada lingkungan Wumpus World berukuran 8×8 dengan satu posisi awal (Start), satu posisi gold sebagai tujuan (Goal), beberapa lubang (Pit), dan beberapa Wumpus sebagai rintangan. Pengujian dilakukan menggunakan lingkungan wumpus world pada gambar 4.1.



Gambar 4.1.

A. Hasil Pengujian

Tabel berikut menunjukkan hasil perbandingan ketujuh algoritma pencarian yang diuji.

Algoritma	Path length	Nodes explored	Execution Time(ms)	Success
DFS	34	36	0.3601	Ya
BFS	14	52	0.2424	Ya
UCS	14	52	0.2602	Ya
Greedy	16	17	0.094	Ya
GBFS	16	17	0.0928	Ya
A*	14	48	0.259	Ya
Beam (w=3)	14	39	0.2356	Ya

Seluruh algoritma berhasil menemukan jalur dari Start menuju Goal. Namun terdapat perbedaan pada kualitas solusi, jumlah node yang dieksplorasi, dan waktu komputasi yang dibutuhkan.

B. Analisis Panjang Jalur

BFS, UCS, A*, dan Beam Search menghasilkan jalur terpendek dengan panjang 14 langkah. Hasil ini menunjukkan bahwa keempat algoritma tersebut mampu menemukan solusi optimal pada konfigurasi lingkungan yang digunakan.

BFS memperoleh solusi optimal karena melakukan eksplorasi berdasarkan level kedalaman. UCS juga menghasilkan solusi yang sama karena setiap perpindahan memiliki biaya yang seragam.

A* mampu menemukan jalur optimal dengan memanfaatkan kombinasi biaya aktual dan heuristik Manhattan Distance. Sementara itu, Beam Search dengan beam width sebesar 3 juga berhasil menemukan jalur optimal karena jumlah kandidat yang dipertahankan masih cukup untuk mengarahkan pencarian menuju solusi terbaik.

Sebaliknya, Greedy Search dan GBFS menghasilkan jalur sepanjang 16 langkah. Jalur tersebut hanya lebih panjang dua langkah dibandingkan solusi optimal. Hal ini terjadi karena kedua algoritma lebih berfokus pada nilai heuristik dibandingkan biaya total jalur.

DFS menghasilkan jalur terpanjang yaitu 34 langkah. Algoritma ini tidak mempertimbangkan optimalitas sehingga cenderung menjelajahi banyak cabang sebelum mencapai tujuan.

C. Analisis Jumlah Node Eksplorasi

Jumlah node yang dieksplorasi menunjukkan tingkat efisiensi proses pencarian.

Greedy Search dan GBFS hanya mengeksplorasi 17 node, menjadikannya algoritma yang paling hemat eksplorasi. Hal ini menunjukkan bahwa penggunaan heuristik Manhattan Distance sangat efektif dalam mengarahkan pencarian menuju tujuan.

Beam Search mengeksplorasi 39 node, lebih sedikit dibandingkan BFS, UCS, maupun A*. Hal ini terjadi karena Beam Search membatasi jumlah kandidat yang dipertahankan pada setiap level pencarian.

A* mengeksplorasi 48 node, lebih sedikit dibandingkan BFS dan UCS yang masing-masing mengeksplorasi 52 node. Dengan demikian, A* mampu mempertahankan optimalitas solusi sambil mengurangi ruang pencarian.

DFS hanya mengeksplorasi 36 node, namun menghasilkan jalur yang jauh lebih panjang. Hasil ini menunjukkan bahwa jumlah node yang sedikit tidak selalu berarti kualitas solusi yang lebih baik.

D. Analisis Waktu Eksekusi

GBFS menjadi algoritma tercepat dengan waktu eksekusi sebesar 0,0928 ms, diikuti oleh Greedy Search sebesar 0,0940 ms. Kedua algoritma tersebut bekerja sangat cepat karena hanya mempertimbangkan informasi heuristik.

Beam Search mencatat waktu eksekusi sebesar 0,2365 ms dan berhasil memperoleh solusi optimal. Hasil ini menunjukkan bahwa Beam Search dapat menjadi alternatif yang baik ketika diperlukan keseimbangan antara kualitas solusi dan efisiensi komputasi.

BFS memiliki waktu eksekusi sebesar 0,2424 ms, sedangkan UCS membutuhkan 0,2602 ms. Perbedaan tersebut disebabkan oleh penggunaan struktur priority queue pada UCS yang menambah overhead komputasi.

A* memiliki waktu eksekusi sebesar 0,2590 ms, hampir setara dengan UCS. Meskipun demikian, A* mengeksplorasi lebih sedikit node sehingga menunjukkan efisiensi pencarian yang lebih baik.

E. Pembahasan

Hasil pengujian menunjukkan bahwa karakteristik masing-masing algoritma sangat memengaruhi performa pencarian.

DFS kurang cocok digunakan ketika tujuan utama adalah menemukan jalur terpendek karena menghasilkan jalur sepanjang 34 langkah. Meskipun jumlah node yang dieksplorasi tidak terlalu besar, kualitas solusi yang diperoleh rendah.

BFS dan UCS berhasil menemukan solusi optimal, tetapi membutuhkan eksplorasi node yang paling banyak. Kedua algoritma ini cocok digunakan ketika optimalitas solusi lebih penting dibandingkan efisiensi pencarian.

Greedy Search dan GBFS menunjukkan performa terbaik dalam hal kecepatan dan jumlah node yang dieksplorasi. Namun, keduanya tidak selalu menghasilkan jalur optimal karena hanya berfokus pada nilai heuristik.

A* memberikan kompromi yang baik antara optimalitas dan efisiensi. Algoritma ini menghasilkan jalur optimal dengan jumlah eksplorasi yang lebih sedikit dibandingkan BFS dan UCS.

Beam Search menunjukkan hasil yang baik. Dengan beam width sebesar 3, algoritma ini berhasil menemukan jalur optimal sepanjang 14 langkah dengan jumlah eksplorasi hanya 39 node. Nilai tersebut lebih rendah dibandingkan BFS, UCS, dan A*. Oleh karena itu, pada skenario pengujian ini Beam Search memberikan kombinasi yang baik antara kualitas solusi dan efisiensi pencarian.

KESIMPULAN DAN SARAN

A. Kesimpulan

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan pada lingkungan Wumpus World berukuran 8×8 , dapat disimpulkan bahwa seluruh algoritma pencarian yang diuji, yaitu Depth-First Search (DFS), Breadth-First Search (BFS), Uniform Cost Search (UCS), Greedy Search, Greedy Best-First Search (GBFS), A*, dan Beam Search, berhasil menemukan jalur dari posisi awal menuju posisi tujuan. Namun demikian, setiap algoritma menunjukkan karakteristik performa yang berbeda dalam hal panjang jalur yang dihasilkan, jumlah node yang dieksplorasi, dan waktu eksekusi.

Hasil pengujian menunjukkan bahwa BFS, UCS, A*, dan Beam Search mampu menemukan jalur optimal dengan panjang 14 langkah. Di sisi lain, Greedy Search dan GBFS menghasilkan jalur sepanjang 16 langkah, sedangkan DFS menghasilkan jalur terpanjang yaitu 34 langkah. Perbedaan tersebut menunjukkan bahwa strategi pencarian yang digunakan sangat memengaruhi kualitas solusi yang diperoleh. Algoritma yang mempertimbangkan biaya pencarian dan heuristik secara lebih terstruktur cenderung menghasilkan solusi yang lebih baik dibandingkan algoritma yang hanya berfokus pada eksplorasi mendalam atau nilai heuristik semata.

Dari aspek efisiensi pencarian, Greedy Search dan GBFS menjadi algoritma yang paling hemat karena hanya mengeksplorasi 17 node serta memiliki waktu eksekusi tercepat, yaitu kurang dari 0,1 milidetik. Meskipun demikian, kedua algoritma tersebut tidak selalu menghasilkan jalur optimal. Sementara itu, BFS dan UCS berhasil menemukan solusi optimal tetapi memerlukan eksplorasi node yang lebih banyak. A* menunjukkan performa yang lebih seimbang karena tetap menghasilkan jalur optimal dengan jumlah eksplorasi yang lebih rendah dibandingkan BFS dan UCS.

Beam Search dengan beam width sebesar 3 juga memberikan hasil yang baik karena mampu menemukan jalur optimal dengan jumlah node yang dieksplorasi lebih sedikit dibandingkan BFS, UCS, dan A*. Oleh karena itu, dapat disimpulkan bahwa A* dan Beam Search merupakan algoritma yang paling seimbang pada skenario pengujian ini karena mampu menghasilkan solusi optimal dengan tingkat efisiensi yang baik. Sementara itu, GBFS menjadi algoritma tercepat dan paling hemat eksplorasi node, meskipun harus mengorbankan optimalitas solusi.

B. Saran

Penelitian ini masih dapat dikembangkan lebih lanjut untuk memperoleh analisis yang lebih mendalam mengenai performa algoritma pencarian. Salah satu pengembangan yang dapat dilakukan adalah dengan menggunakan ukuran grid yang lebih besar dan konfigurasi lingkungan yang lebih kompleks sehingga perbedaan performa antar algoritma dapat terlihat secara lebih signifikan. Selain itu, jumlah dan posisi rintangan dapat dibuat lebih bervariasi untuk menguji kemampuan algoritma dalam menghadapi tingkat kesulitan yang berbeda.

Pengembangan selanjutnya juga dapat dilakukan dengan menambahkan algoritma pencarian lain, seperti Iterative Deepening Search, Bidirectional Search, atau algoritma berbasis optimasi dan metaheuristik. Pengujian Beam Search dengan berbagai nilai beam width juga menarik untuk dilakukan guna mengetahui pengaruh parameter tersebut terhadap kualitas solusi dan efisiensi pencarian. Selain itu, lingkungan Wumpus World dapat dibuat lebih realistis dengan menambahkan elemen persepsi agen, seperti Breeze dan Stench, sehingga proses pengambilan keputusan tidak hanya bergantung pada pencarian jalur tetapi juga pada kemampuan inferensi terhadap kondisi lingkungan. Dengan pengembangan tersebut, hasil penelitian diharapkan dapat memberikan gambaran yang lebih komprehensif mengenai penerapan berbagai strategi algoritma pencarian dalam menyelesaikan permasalahan pencarian jalur.

LAMPIRAN

<https://github.com/haikalimamridha/wumpus>

REFERENCES

- [1] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2021.
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

PERNYATAAN

Dengan ini saya menyatakan bahwa makalah yang saya susun merupakan hasil karya saya sendiri dan tidak merupakan saduran, terjemahan, maupun plagiasi dari karya orang lain. Seluruh sumber yang digunakan telah dicantumkan secara benar sesuai dengan kaidah penulisan ilmiah.

Terkait penggunaan teknologi kecerdasan buatan (Artificial Intelligence), saya menyatakan bahwa (hapus salah satu):

Saya **menggunakan** alat bantu AI sebagai pendukung dalam penyusunan makalah ini dengan rincian sebagai berikut:

- [3] Nama alat AI: chatgpt, claude ai
- [4] Tujuan penggunaan: membantu proses penyusunan makalah, pencarian referensi, pemahaman materi, pengembangan dan perbaikan kode program, serta analisis hasil pengujian.
- [5] Ruang lingkup penggunaan: Ruang lingkup penggunaan: digunakan untuk membantu penyusunan struktur dan tata bahasa makalah, memberikan penjelasan konsep algoritma pencarian membantu proses pengembangan dan perbaikan kode program

Penggunaan AI, apabila ada, hanya bersifat sebagai alat bantu dan tidak menggantikan peran saya sebagai penulis utama. Seluruh isi, analisis, serta kesimpulan dalam makalah ini tetap merupakan tanggung jawab saya sepenuhnya

Semarang, 7 Juni 2026



Haikal Imam Ridha
24060124130097